DBMS – SALES AND CUSTOMER ANALYTICS SYSTEM

Project Title: Lenskart E-Commerce Order Management System

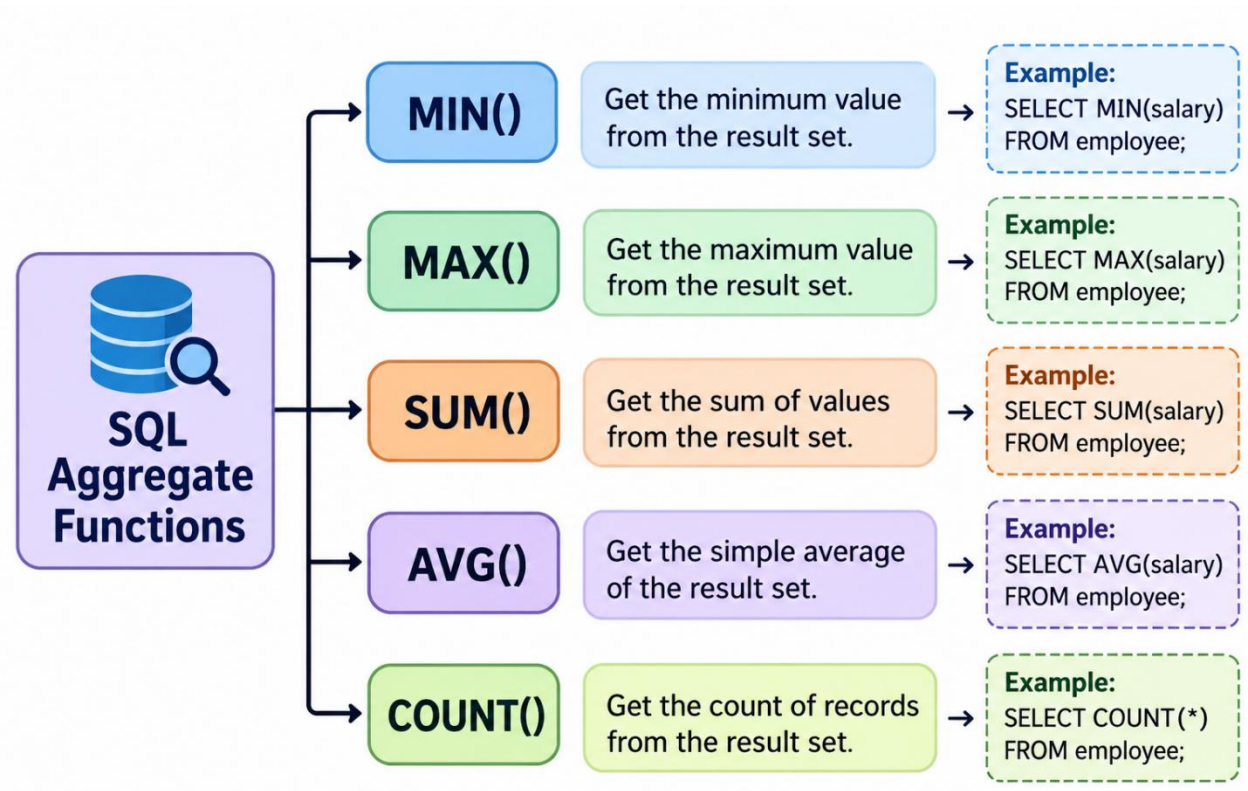
Objectives

1. Apply COUNT(),SUM(),AVG(),MIN(), and MAX().
2. Generate total sales reports.
3. Find top customers based on purchase amount.
4. Identify best-selling products.
5. Perform category-wise sales analysis.

1. Aggregate Functions

Definition

Aggregate functions are SQL functions used to perform calculations on a group rows and return a single result.



1.1 COUNT()

Find the total number of orders :

```
SQL> SELECT COUNT(Order_ID) AS Total_Orders
  2  FROM Orders;

TOTAL_ORDERS
------------
          10
```

1.2 SUM()

Find the total sales amount :

```
SQL> SELECT SUM(Total_Amount) AS Total_Sales
  2  FROM Orders;

TOTAL_SALES
-----------
       24490
```

1.3 AVG()

Find the average order amount :

```
SQL> SELECT AVG(Total_Amount) AS Average_Order
  2  FROM Orders;

AVERAGE_ORDER
-------------
         2449
```

1.4 MIN()

Find the lowest order amount :

```
SQL> SELECT MIN(Total_Amount) AS Minimum_Order
  2  FROM Orders;

MINIMUM_ORDER
-------------
         1299
```

1.5 MAX()

Find the highest order amount :

```
SQL> SELECT MAX(Total_Amount) AS Maximum_Order
  2  FROM Orders;

MAXIMUM_ORDER
-------------
         3999
```

2. Generate Total Sales Reports

A sales report can be generated using the SUM() function on the Total_Amount column.

```
SQL> SELECT
  2      COUNT(Order_ID) AS Total_Orders,
  3      SUM(Total_Amount) AS Total_Sales,
  4      AVG(Total_Amount) AS Average_Order,
  5      MIN(Total_Amount) AS Minimum_Order,
  6      MAX(Total_Amount) AS Maximum_Order
  7  FROM Orders;

TOTAL_ORDERS TOTAL_SALES AVERAGE_ORDER MINIMUM_ORDER MAXIMUM_ORDER
------------ ----------- ------------- ------------- -------------
          10       24490          2449          1299          3999
```

The output above is based on the sample Orders data used in your EcoCart project.

2.1 Customer-wise Sales Report

To find how much each customer has purchased.

```
SELECT
    C.Customer_ID,
    C.Customer_Name,
    SUM(O.Total_Amount) AS Total_Purchase
FROM Customer C
JOIN Orders O
ON C.Customer_ID = O.Customer_ID
GROUP BY C.Customer_ID, C.Customer_Name
ORDER BY Total_Purchase DESC;

CUSTOMER_ID CUSTOMER_NAME              TOTAL_PURCHASE
----------- ------------------------- --------------
         10 Harini                              3999
          5 Vignesh                             3499
          3 Karthik                             2999
          9 Sanjay                              2699
          1 Arun Kumar                          2499
          7 Rahul                               2199
          2 Priya                               1899
          8 Nandhini                            1799
          4 Divya                               1599
          6 Keerthana                           1299

10 rows selected.
```

Purpose

This report shows

- Customer ID
- Customer name
- Total purchase amount
- Customers arranged according to purchase amount

3. Find Top Customers Based on Purchase amount

The SUM() function can be combined with GROUP BY and ORDER BY to identify customers with higher purchase amounts.

```
SELECT
    C.Customer_ID,
    C.Customer_Name,
    SUM(O.Total_Amount) AS Total_Purchase
FROM Customer C
JOIN Orders O
ON C.Customer_ID = O.Customer_ID
GROUP BY C.Customer_ID, C.Customer_Name
ORDER BY Total_Purchase DESC;

CUSTOMER_ID CUSTOMER_NAME               TOTAL_PURCHASE
----------- ------------------------------------------
         10 Harini                                3999
          5 Vignesh                               3499
          3 Karthik                               2999
          9 Sanjay                                2699
          1 Arun Kumar                            2499
          7 Rahul                                 2199
          2 Priya                                 1899
          8 Nandhini                              1799
          4 Divya                                 1599
          6 Keerthana                             1299

10 rows selected.
```

The customer appearing at the top has the highest purchase amount in the generated report.

4. Identify Best-Selling Products

The Order_Details table contains the quantity of each product ordered.

Therefore, we can use SUM(Quantity) to find the total quantity sold for each product.

```
SELECT
    P.Product_ID,
    P.Product_Name,
    SUM(OI.Quantity) AS Total_Quantity_Sold
FROM Product P
JOIN Order_Item OI
ON P.Product_ID = OI.Product_ID
GROUP BY P.Product_ID, P.Product_Name
ORDER BY Total_Quantity_Sold DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
       203
Reading Glasses
                 5

       201
Classic Eyeglasses
                 4
```

Purpose

This query :

1. Connects Product and Order_Details.
2. Groups records according to product.
3. Calculates total quantity sold.
4. Displays products from highest to lowest quantity sold.

4.1 Product Sales Amount

Instead of only calculating quantity, we can calculate the total sales generated by each product.

```
SELECT
    P.Product_ID,
    P.Product_Name,
    SUM(OI.Quantity) AS Total_Quantity_Sold,
    SUM(OI.Quantity * P.Price) AS Total_Sales
FROM Product P
JOIN Order_Item OI
ON P.Product_ID = OI.Product_ID
GROUP BY P.Product_ID, P.Product_Name
ORDER BY Total_Sales DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_SALES
------------------- -----------
       201
Classic Eyeglasses
                  4        4800


       203
Reading Glasses
                  5        4000

PRODUCT_ID
----------
PRODUCT_NAME
```

This gives both quantity sold and sales amount.

5. Category-wise Sales Analysis

Category-wise analysis helps determine the sales generated by each product category

```
SELECT
    C.Category_ID,
    C.Category_Name,
    SUM(OI.Quantity) AS Total_Quantity_Sold,
    SUM(OI.Quantity * P.Price) AS Total_Sales
FROM Category C
JOIN Product P
ON C.Category_ID = P.Category_ID
JOIN Order_Item OI
ON P.Product_ID = OI.Product_ID
GROUP BY C.Category_ID, C.Category_Name
ORDER BY Total_Sales DESC;

CATEGORY_ID
-----------
CATEGORY_NAME
--------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_SALES
------------------- -----------
         1
Eyeglasses
                 12       12700


         2
Sunglasses
                  2        3000

CATEGORY_ID
```

6. Combined Business Sales Report

A complete sales report can combine customer, order, product, and category information.

```
SELECT
    C.Customer_Name,
    O.Order_ID,
    P.Product_Name,
    CAT.Category_Name,
    OI.Quantity,
    (OI.Quantity * P.Price) AS Subtotal
FROM Customer C
JOIN Orders O
ON C.Customer_ID = O.Customer_ID
JOIN Order_Item OI
ON O.Order_ID = OI.Order_ID
JOIN Product P
ON OI.Product_ID = P.Product_ID
JOIN Category CAT
ON P.Category_ID = CAT.Category_ID
ORDER BY O.Order_ID;


CUSTOMER_NAME                                      ORDER_ID
-------------------------------------------------- ----------
PRODUCT_NAME
--------------------------------------------------------------------
CATEGORY_NAME
--------------------------------------------------------------------
  QUANTITY   SUBTOTAL
---------- ----------
Arun Kumar                                            101
Classic Eyeglasses
```

Report Structure

This report can be used to understand:

- Who purchased the product
- Which order was placed
- Which product was purchased
- Which category the product belongs to
- Quantity purchased
- Amount generated